

**Autonomous Multi-Agent Curriculum Ingestion and Dual Training Kit Generator:
Multi-Model LLM Integration, Code Synthesis, and Four-Gate Verification**

A Graduate Degree Research & Engineering Project Proposal — Pre-Implementation Specification

Project Parameter	Proposed Specification & Architectural Attributes
Project Title	Autonomous Multi-Agent Curriculum Ingestion and Dual Training Kit Generator
Academic Context	COSC726: Agentic Artificial Intelligence Proposer: Mohammad Abulgasim Supervisor: Dr. Fakhreldeen Saeed
Architecture & Topology	Sequential & Hierarchical Multi-Agent Topology, Dual-Granularity Parent-Child RAG, Multi-Model Seam
Model Inference Seam	Vendor-Agnostic ModelClient: Local Qwen 2.5 (1.5B via Ollama), Cohere Command-R, Gemini, GPT-4o, Claude, Mock
Verification Engine	The Four Validation Gates Framework (Parses, Conforms, Refers, Coheres) with Autonomous Critic Self-Healing
Target Deliverables	Instructor Master Kit (PDF & PPTX 16:9) + Trainee Learning Kit (PDF & PPTX 16:9) with verified code & assessments

1. Executive Summary & Project Abstract

Academic universities and technical workforce academies face persistent friction when converting raw, unstructured course syllabi and textbook outlines into pedagogically sound, calibrated instructional packages. Conventional single-turn Large Language Model (LLM) prompts fail catastrophically in this domain—exhibiting arithmetic drift in contact hours, context window truncation, uncalibrated assessments with ghost citations, non-executable code snippets, and a complete absence of pedagogical differentiation between instructor delivery manuals and student learning workbooks.

This project proposes an end-to-end autonomous multi-agent AI system that ingests unstructured curriculum files (PDF/TXT), deterministically computes instructional hours, and orchestrates specialized

agents to synthesize dual, publication-grade training kits: the Instructor Master Kit (PDF/PPTX) and the Trainee Learning Kit (PDF/PPTX). By establishing an architectural separation between model reasoning and programmatic invariants ('Compute-in-Code'), deploying dual-granularity Parent-Child RAG, validating code via Abstract Syntax Trees (AST), and enforcing the Four Validation Gates, the proposed system guarantees 100% mathematical, syntactic, and citation compliance.

★ Project Thesis & Core Innovation

By enforcing deterministic code invariants (pure Python schedule computation and AST compilation) and a Four-Gate verification loop, autonomous multi-agent systems can eliminate the hallucinations, arithmetic drift, and formatting breakdowns of probabilistic LLMs, delivering production-ready academic training packages without human intervention.

2. Problem Statement & Research Motivation

Modern curriculum engineering demands synchronized instructional alignment. However, deploying naive, single-turn LLMs to generate curriculum materials introduces critical operational failure modes:

- **Arithmetic Drift & Temporal Hallucination:** LLMs cannot guarantee mathematical summation closure; when allocating contact hours across weeks, weekly totals routinely diverge from institutional course targets.
- **RAG Chunking Dilemma & Context Amputation:** Standard fixed-size text chunking splits essential syllabus topic lists and rubrics mid-sentence, blinding downstream generation agents to contextual prerequisites.
- **Syntactically Invalid & Untested Code:** Generative models produce pseudo-code or broken scripts with missing imports, undeclared variables, or syntax errors that disrupt student learning.
- **Uncalibrated Assessments & Ghost Citations:** LLM-generated quizzes suffer from obvious distractors, ambiguous answer keys, and fabricated citations that do not exist in the source syllabus.
- **The One-Size-Fits-All Fallacy:** Single prompts produce a single generic narrative, failing to distinguish between instructor teaching guides (solutions, cues, rubrics) and student workbooks (TODOs, self-checks).

3. Research Questions & Specific Proposed Objectives

This research project investigates five core academic questions and targets five technical objectives:

- **RQ1 & SRO1 (Model Seam):** How can inference be abstracted via an agnostic ModelClient seam to allow hot-swapping between local private models (Ollama Qwen 2.5) and cloud APIs without altering agent logic?
- **RQ2 & SRO2 (Parent-Child RAG):** How effectively does dual-granularity indexing (~200 char child targets dereferenced to ~700 char parent containers) eliminate context truncation and ghost citations?
- **RQ3 & SRO3 (Compute-in-Code):** Can delegating contact hour arithmetic strictly to deterministic Python algorithms guarantee zero temporal drift ($| \text{Actual} - \text{Target} | = 0.0h$) across arbitrary course durations?
- **RQ4 & SRO4 (Code Synthesis & AST):** How can dynamic LLM program synthesis be bounded by Abstract Syntax Tree (AST) validation to ensure 100% syntactically valid starter and solution scripts?
- **RQ5 & SRO5 (Four-Gate Verification):** What is the self-healing convergence rate of an autonomous CriticAgent when repairing injected defects across JSON syntax, quotas, provenance, and code?

4. Theoretical Foundations & Architectural Principles

The proposed system is grounded in four foundational software engineering and pedagogical principles:

(1) Multi-Agent Role Specialization: Decomposing monolithic prompts into discrete, bounded agents (Ingestion, Planning, Theory, Lab, Assessment, Critic); (2) Separation of Model, Harness, and Agent: Treating LLMs strictly as stateless token generators while maintaining execution state in Python; (3) Compute-in-Code Determinism: Fully removing arithmetic calculations from probabilistic generation; and (4) Cognitive Load Theory & Pedagogical Differentiation: Scaffolding student learning with active problem-solving (TODO starter templates) while equipping instructors with comprehensive delivery guidance, timing cues, and full solutions.

5. Proposed Multi-Agent System Architecture

The proposed architecture operates as a sequential-hierarchical multi-agent pipeline governed by Pydantic V2 data contracts. The system coordinates specialized autonomous agents across bounded operational domains:

Specialist Agent	Subsystem Domain	Input Contract / Triggers	Core Autonomous Responsibilities
SyllabusParser & Ingestion	Document Preprocessing	Raw PDF / TXT Syllabus File	Extracts clean text, detects language (EN/AR), course code, duration, and lab presence.
ParentChildChunker	Dual-Granularity RAG	Extracted Document Stream	Segments text into ~700 char parent blocks and ~200 char child targets with citation tags.
ScheduleMathEngine	Deterministic Scheduler	Weeks (W) & Total Hours (H)	Calculates exact weekly lecture/lab allocations via Python ('Compute-in-Code'); zero drift.
ModuleDecomposer	Instructional Planner	KnowledgeBase & Schedule	Decomposes curriculum into weekly titles, topics, and lab assignments grounded in text.
TheorySpecialistAgent	Pedagogical Synthesis	Weekly Topic Decomposition	Synthesizes instructor lecture scripts, discussion cues, student reading syntheses, and outcomes.
PracticalLabSpecialist	Code & Lab Synthesis	Decomposition & Domain Tool	Generates executable starter code (TODOs), debugging tasks, solution code, and grading rubrics.
AssessmentSpecialist	Evaluation Synthesis	Grounded Syllabus Context	Synthesizes strictly 5 MCQs (4 options) and 5 True/False questions with rationales and citations.
CriticAgent & Verifier	Quality & Self-Healing	Raw Weekly Module Package	Audits payload against Four Validation Gates; triggers targeted re-prompting if defects occur.
Dual Exporters (PDF/Deck)	Document Compilation	Verified Module Packages	Renders publication-ready PDF kits (ReportLab) and 16:9 widescreen slide decks (python-pptx).

6. The Four Validation Gates Verification Framework

Quality assurance is mathematically formulated through four sequential validation gates. A weekly package $\mathcal{P}_w$ is certified for publication if and only if its composite score

$G_{\text{composite}} = \frac{1}{4}(G_1 + G_2 + G_3 + G_4) = 1.00$:

Validation Gate	Target Dimension	Formal Verification Mechanism	Acceptance Invariant
Gate 1 (Parses)	Syntactic Integrity	Validates that all LLM output strings parse into strict JSON serializable by Pydantic V2 schemas.	100% Valid JSON ($G_1 = 1.0$)
Gate 2 (Conforms)	Structural Quotas	Enforces strictly 5 MCQs (4 options, valid key) and 5 True/False items per week; starter TODOs.	Exact Quota Match ($G_2 = 1.0$)
Gate 3 (Refers)	Citation Grounding	Cross-references all cited IDs against KnowledgeBase registry; eliminates ghost citations.	Zero Ghost Citations ($G_3 = 1.0$)
Gate 4 (Coheres)	Temporal & Code Math	Reconciles hour sums ($\sum h_i = H$); validates Python scripts via Abstract Syntax Tree (<code>`ast.parse`</code>).	Zero Hour Drift & Valid AST ($G_4 = 1.0$)

7. Proposed Phased Work Breakdown Structure (WBS)

To ensure modular verification and rigorous engineering, the proposed implementation is partitioned into six self-contained, sequential phases:

Phase	Milestone Title	Core Technical Tasks	Planned Code Artifacts	Validation Gate
Phase 1	Architecture & Contracts	Define Pydantic V2 schemas, ModelClient seam, and Gate 1-4 verification logic.	contracts.py, validation_gates.py, model_client.py	Gate 1 (Parses) & Fault Injection
Phase 2	Ingestion & Parent-Child RAG	Develop PDF/TXT parser, semantic paragraph chunker (~700/200 chars), KnowledgeBase.	pdf_parser.py, parent_child_chunker.py, knowledge_base.py	Gate 3 (Refers) & Citation Audit
Phase 3	Instructional Planning	Implement ScheduleMathEngine ('Compute-in-Code') and ModuleDecomposer.	schedule_math_engine.py, module_decomposer.py	Gate 4 (Hour Drift = 0.0h)
Phase 4	Content & Real Code Engine	Build TheorySpecialistAgent, PracticalLabSpecialistAgent, and AST Python compiler.	theory_agent.py, lab_specialist_agent.py	Gate 4 (AST Code Compilation)
Phase 5	Assessment & Critic Agent	Construct AssessmentSpecialistAgent (5 MCQs + 5 T/F) and CriticAgent self-healing.	assessment_agent.py, critic_agent.py	Gate 2 (Quotas) & Self-Healing
Phase 6	Publication Exporters & UI	Build ReportLab PDF exporter, 16:9 PPTX exporter, Streamlit UI dashboard, and CLI.	pdf_exporter.py, pptx_exporter.py, app.py, cli.py	End-to-End Pipeline Conformance

8. Anticipated Tangible Deliverables & Pedagogical Differentiation

The proposed pipeline produces four distinct, publication-grade artifacts in PDF and PPTX formats, enforcing strict pedagogical differentiation:

Instructional Dimension	Instructor Master Kit (PDF & PPTX 16:9)	Trainee Learning Kit (PDF & PPTX 16:9)
Target Audience & Role	Course Instructors & Lecturers: Delivery Manual & Presentation Deck	Enrolled Students & Trainees: Active Study Guide & Lab Workbook
Lecture Delivery Scripts	Full delivery script with vocal emphasis notes, pedagogical cues, and timing markers.	Scaffolded reading synthesis, conceptual summaries, and weekly learning outcomes.
Classroom Discussions	Discussion questions paired with complete model answer keys and common student pitfalls.	Open reflection prompts without answers to drive collaborative peer debate.
Practical Lab Code	Production-grade reference implementation with assertions and grading point rubrics.	Student starter template with function stubs, explicit TODO tasks, and debug challenges.
Formative Assessments	5 MCQs + 5 T/F items per week with correct answer keys, rationales, and citation provenance.	5 MCQs + 5 T/F items formatted as an unsolved self-evaluation quiz (answers redacted).
Document Presentation	Professional navy ReportLab PDF + 16:9 Widescreen slide deck with instructor notes.	Student workbook PDF with code boxes + 16:9 study deck focused on core concepts.

9. Proposed Evaluation Protocol & Acceptance Criteria

System performance will be validated through automated conformance test suites across three experimental vectors: (1) Baseline Conformance Auditing across all four gates; (2) Synthetic Fault Injection (corrupting schemas, citations, and syntax) to verify CriticAgent self-healing recovery; and (3) Multi-Domain Benchmark Testing across English/Arabic and Theory/Practical syllabi.

Evaluation Metric	Target Acceptance Threshold	Verification Method & Tool	Proposed Impact
Gate 1 Pass Rate	100.0% (Zero JSON Syntax Errors)	Automated Pydantic V2 Schema Validation	Guarantees robust pipeline data contracts
Gate 2 Quota Precision	100.0% (Strictly 5 MCQs + 5 T/F)	Structural Assertions in FourGatesEngine	Prevents assessment bank omissions
Gate 3 Citation Recall	100.0% (Zero Ghost Citations)	KnowledgeBase Citation Registry Dereferencing	Eliminates factual hallucinations
Gate 4 Hour Sum Invariance	0.00 Hours Drift ($ \text{Sum} - \text{Target} = 0$)	ScheduleMathEngine Programmatic Assertions	Enforces institutional accreditation rules
Gate 4 AST Code Validity	100.0% Valid AST Compilations	Python Native <code>`ast.parse()`</code> Verification	Guarantees executable, bug-free lab code
CriticAgent Self-Healing	100.0% Recovery on Defect Injections	Automated Fault Injection Harness	Ensures fully autonomous error recovery

10. Proposed Implementation Schedule, Risk Management & Conclusion

The project is structured across a 14-week timeline: Weeks 1-2 (Phase 1), Weeks 3-4 (Phase 2), Weeks 5-6 (Phase 3), Weeks 7-8 (Phase 4), Weeks 9-10 (Phase 5), Weeks 11-12 (Phase 6), and Weeks 13-14 (Empirical Evaluation & Final Defense).

Anticipated Technical Risk	Severity	Proposed Mitigation Strategy
Model Arithmetic Hallucination	High	Enforce 'Compute-in-Code' strictly in Python; prohibit LLM from calculating contact hours.
Syllabus Context Truncation	High	Adopt Parent-Child RAG; retrieve complete ~700 char parent units and adjacent continuation chunks.
Cloud API Latency / Rate Limits	Medium	Implement abstracted ModelClient with local offline Qwen 2.5 (Ollama) and deterministic Mock seams.
Syntactically Broken Code	High	Compile all generated starter/solution code via Python AST; reject and re-prompt on syntax errors.
Arabic RTL Typography Breakdowns	Medium	Utilize arabic_reshaper and python-bidi with registered TrueType fonts (Arial/Tahoma).

Academic Conclusion: Conclusion: This proposed system establishes a rigorous, production-grade paradigm for educational curriculum engineering. By wedding multi-agent role specialization, deterministic temporal mathematics, dual-granularity RAG, and Four-Gate automated verification, the project bridges the gap between probabilistic generative AI and accredited educational publishing.